Machine Learning Lab Assignment

Program: M.Tech – 1st Year Course: Machine Learning Type: Practical / Lab Assignment

Topic: Forward Propagation and Backpropagation in Neural Networks

Total Questions: 2 (Section A: 1, Section B: 1) Date: 11 November 2026

General Instructions

- Use Python with NumPy, Pandas, Matplotlib, and scikit-learn. Implement every forward pass, backward pass, and parameter update from its formula using NumPy only (no TensorFlow, PyTorch, or automatic differentiation); use scikit-learn only for datasets, splits, scalers, and wherever a question asks you to verify your result.
- For reproducibility, use random_state = 42 for all datasets, data splits, and scikit-learn models. Report the Python and package versions used.
- Show the hand calculations asked for in Questions 1(a) and 2(a) with every intermediate value (typed, or neatly handwritten and scanned). Report forward-pass values to 4 decimal places and gradients to 6 decimal places.
- Every figure must have a title, labelled axes, and a legend where more than one curve is plotted.
- Add a short "Observations / Inference" note (2-3 sentences) after each question interpreting your results.
- Both questions are compulsory.
- Copied code or shared solutions between students will be treated as plagiarism.

Section A: Forward Propagation (Question 1)

Question 1: Forward Propagation in a Feed-Forward Neural Network

Dataset / Setup (Hand-calculation network): A fully connected 2–2–1 network (2 inputs, 2 hidden neurons, 1 output neuron). Layers are numbered from 1, and row j of $W^{(l)}$ holds the weights into neuron j of layer l. The forward pass is:

$$a^{(0)} = x, \qquad z^{(l)} = W^{(l)}\, a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f\left(z^{(l)}\right), \qquad \hat{y} = a^{(2)}$$

The input, the target, and the parameters of the network are:

$$x = \begin{bmatrix} 0.6 \\ 0.9 \end{bmatrix}, \qquad y = 1$$

$$W^{(1)} = \begin{bmatrix} 0.2 & -0.3 \\ 0.4 & 0.1 \end{bmatrix}, \qquad b^{(1)} = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}$$

$$W^{(2)} = [0.5 \quad -0.6], \qquad b^{(2)} = 0.2$$

Unless a task says otherwise, every neuron uses the sigmoid activation, and the loss of one sample is the squared error:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \qquad L = \frac{1}{2}(y - \hat{y})^2$$

Dataset / Setup (Scikit-learn network): Breast Cancer Wisconsin dataset (load_breast_cancer; 569 samples, 30 features) with target = 1 as the positive class. Use a 70/30 train–test split (stratify = y, random_state = 42),

standardize with StandardScaler fitted on the training set only, and train MLPClassifier(hidden_layer_sizes = (8, 4), activation = 'relu', max_iter = 500, random_state = 42).

Tasks

(a) By hand, compute $z^{(1)}$, $a^{(1)}$, $z^{(2)}$, $\hat{y}$, and L for the given input, showing every intermediate step and reporting values to 4 decimal places. Then reproduce the same values in NumPy using the matrix form $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$, and also compute the binary cross-entropy of this prediction:

$$-[\, y\ln\hat{y} + (1-y)\ln(1-\hat{y})]$$

(b) Implement a vectorized function forward(X, W1, b1, W2, b2, hidden_activation) that processes N samples stored as the rows of X. For the batch

$$X = \begin{bmatrix} 0.6 & 0.9 \\ 0.1 & 0.2 \\ -0.5 & 0.4 \\ 1.0 & -1.0 \\ 0.0 & 0.0 \end{bmatrix}$$

verify with np.allclose that the batched outputs equal the outputs obtained by passing the five samples through the network one at a time. Then, for $x = (0.6,\ 0.9)^T$ with the weights unchanged, tabulate $\hat{y}$ and L when the hidden activation is sigmoid, tanh, or $\mathrm{ReLU}(z) = \max(0,\ z)$ (the output neuron stays sigmoid).

(c) On the scikit-learn network, use only the fitted attributes coefs_ and intercepts_ to write your own forward pass (two ReLU hidden layers and a sigmoid output neuron) for the test set, and verify it with np.allclose(..., atol = 1e-8) against predict_proba(X_test)[:, 1]. Report the shape of every weight matrix, the total number of trainable parameters, and the test accuracy (class 1 is predicted when the probability is at least 0.5).

Section B: Backpropagation (Question 2)

Question 2: Backpropagation, Gradient Checking, and Gradient Descent

Dataset / Setup (Network): The same 2–2–1 network, input, target, weights, sigmoid activations, and loss as in Question 1(a). The error term of layer l is defined as:

$$\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}}$$

The network has nine parameters in total: four in $W^{(1)}$, two in $b^{(1)}$, two in $W^{(2)}$, and one in $b^{(2)}$.

Dataset / Setup (Gradient checking and descent): The numerical gradient of a parameter θ is the central difference below, with $h = 10^{-5}$, changing one parameter at a time:

$$g_n = \frac{L(\theta + h) - L(\theta - h)}{2h}$$

The relative error between the analytic gradient g_a and the numerical gradient g_n is:

$$\frac{|g_a - g_n|}{|g_a| + |g_n|}$$

Gradient descent with learning rate η updates all nine parameters simultaneously:

$$\theta \leftarrow \theta - \eta\, \frac{\partial L}{\partial \theta}$$

Tasks

(a) Using the chain rule and $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$, derive expressions for the following quantities and evaluate each by hand at the given weights, reporting values to 6 decimal places:

$$\delta^{(2)}, \quad \delta^{(1)}, \quad \frac{\partial L}{\partial W^{(2)}}, \quad \frac{\partial L}{\partial b^{(2)}}, \quad \frac{\partial L}{\partial W^{(1)}}, \quad \frac{\partial L}{\partial b^{(1)}}$$

(b) Implement the backward pass in NumPy and check that it reproduces your hand-computed gradients. Then verify all nine gradients with the central-difference method and present a table listing each parameter, its analytic gradient, its numerical gradient, and the relative error; the maximum relative error must be below 10^{-7}.

(c) Starting from the given weights, run 2000 gradient-descent updates for each learning rate $\eta \in \{0.1,\ 0.5,\ 2.0\}$, and plot L against the number of updates for all three in one figure (logarithmic y-axis). For each η, report L and $\hat{y}$ after the first update, the smallest number of updates after which $\hat{y} \geq 0.9$, and the final L after 2000 updates.